Complete Pandas Guide (Hinglish) — Keith Galli / freeCodeCamp Tutorial

Based on: "Complete Python Pandas Data Science Tutorial" by Keith Galli (2024 updated edition)
Datasets used in video: coffee.csv (warmup), Olympics dataset (bios.csv, results.csv/results.parquet, noc_regions.csv)

0. Pandas Kya Hai Aur Kyun Zaroori Hai

Pandas ek Python library hai jo tabular data (rows aur columns wala data — jaise Excel sheet) ko handle karne ke liye banayi gayi hai. Tu already OOP jaanta hai, toh soch le:

- **DataFrame:** Ek class ka object jisme data 2D table (***rows* × *columns***) me stored hota hai — bilkul Excel sheet ki tarah.
- **Series:** DataFrame ka ek single column — 1D array jaisa, but with an index (labels) attached.

> **Analogy:** Agar DataFrame ek poori Excel spreadsheet hai, toh Series uski ek single column hai. Aur index un row-labels ko bolte hain jo row ko identify karte hain (jaise Excel me row number, but yaha customizable hota hai).

Installation:

```
pip install pandas numpy openpyxl pyarrow
```

(openpyxl Excel files ke liye, pyarrow fast backend aur parquet files ke liye chahiye.)

Import convention (hamesha yahi use hota hai):

```
import pandas as pd
import numpy as np
```

1. DataFrame Basics — Banate Kaise Hain

DataFrame manually bhi bana sakte ho, list-of-lists ya dictionary se.

```
# List of lists se
df = pd.DataFrame(
    [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]],
    columns=["A", "B", "C"],
    index=["x", "y", "z", "zz"]
)
print(df)
```

Output samajhne ka tarika: rows ke labels (x, y, z, zz) index hain, columns ke naam A, B, C hain — matlab yeh ek 2D grid hai jisme do axes hote hain:

- ***axis=0*** → rows ke direction me (top-to-bottom) — jaise NumPy me axis=0 columns ko collapse karta row-wise.
- ***axis=1*** → columns ke direction me (left-to-right).

> **Analogy (NumPy jaisa hi):** ***axis=0*** ka matlab hai "har column ke andar neeche jao", ***axis=1*** ka matlab "har row ke andar right jao". Jab tu `.sum(axis=0)` karega, toh output ek row banegi (column-wise totals), aur `.sum(axis=1)` se ek column banegi (row-wise totals).

Series banane ka tarika (single column jaisa):

```
s = pd.Series([1, 2, 3], name="my_series")
```

2. Data Load Karna (CSV, Excel, Parquet, TXT)

Real projects me data files se load hota hai, hath se nahi banaya jata.

```
# CSV
coffee = pd.read_csv('./warmup-data/coffee.csv')

# Excel (sheet_name specify karna padta hai agar multiple sheets hain)
olympics_data = pd.read_excel('./data/olympics-data.xlsx', sheet_name="results")

# Parquet (large data ke liye fast, columnar binary format)
results = pd.read_parquet('./data/results.parquet')

# TXT / tab-separated
df_txt = pd.read_csv('./data/file.txt', delimiter='     ')
```

Parquet kyun use karte hain? CSV text-based hota hai (slow, bada file size). Parquet binary + columnar format hai — data science me bade datasets ke liye read/write bahut fast hota hai, aur data types bhi preserve rehte hain (CSV me sab kuch string ban jata hai jab tak tu specify na kare).

3. Data Dekhna Aur Sort Karna (Viewing/Sorting)

```
coffee.head()        # pehli 5 rows
coffee.head(10)      # pehli 10 rows
coffee.tail(2)        # last 2 rows
coffee.sample(5)      # random 5 rows (testing ke liye achha)

# Sorting
coffee.sort_values('Units Sold')                    # ascending default
coffee.sort_values('Units Sold', ascending=False)      # descending
coffee.sort_values(['Coffee Type', 'Units Sold'], ascending=[True, False])  # multi-
column sort
```

> **Analogy:** `sort_values` Excel ke "Sort A→Z" button jaisa hai, bas multiple columns pe ek saath bhi laga sakte ho — priority left-se-right hoti hai (pehle Coffee Type se sort, phir usi group ke andar Units Sold se).

4. DataFrame Ke Baare Me General Info

```
df.columns          # column names ki list
df.index.tolist()   # index labels ki list
df.info()           # dtype, non-null counts, memory usage — sabse zyada use hone wala
command
df.describe()       # numeric columns ka statistical summary (mean, std, min, max,
quartiles)
df.nunique()        # har column me kitne unique values hain
df['A'].unique()    # ek specific column ke unique values
df.shape            # (rows, columns) tuple
df.size             # total elements (rows * columns)
```

> **.info()** ko sabse pehle chalao jab bhi naya dataset load karo — yeh batata hai kaunse column me missing values hain aur dtype kya hai (int, float, object/string, etc.).

5. Indexing Aur Selecting — loc vs iloc

Yeh sabse confusing part hota hai beginners ke liye, isliye analogy se samjho:

- **iloc:** integer location. Bilkul Python list slicing jaisa — position (0, 1, 2...) se access karta hai, chahe index label kuch bhi ho.
- **loc:** label location. Actual index/column ke naam (label) se access karta hai.

```
df.iloc[0]           # first row (position-based)
df.iloc[0:3]         # first 3 rows (position-based, end exclusive)
df.iloc[0, 1]         # row 0, column 1 (position, position)
df.iloc[0:3, 1:3]     # rows 0-2, columns 1-2

df.loc[0]             # row jiska index-label '0' hai
df.loc[0:3]           # label 0 se label 3 TAK (inclusive! yeh iloc se different hai)
df.loc[0:3, 'A':'B']  # rows label 0-3, columns 'A' se 'B' tak
df.loc[:, 'A']        # saari rows, sirf column 'A'
```

Important gotcha: `loc` me slicing inclusive hoti hai (end bhi shamil hota hai), jabki `iloc` me Python ke normal slicing jaisa exclusive hota hai (end shamil nahi hota). Yeh ek common confusion point hai.

Column select karna direct bhi kar sakte ho:

```
coffee['Units Sold']          # single column (Series)
coffee[['Units Sold', 'Day']] # multiple columns (DataFrame) — double brackets note
karo
```

Value set karna:

```
df.loc[1, 'Units Sold'] = 10     # ek specific cell update
df.iloc[1, 2] = 10                # position se update
```

6. Conditional Selection / Filtering (Sabse Important Skill)

Yeh SQL ke WHERE clause jaisa kaam karta hai. Ek boolean condition banti hai (True/False mask) aur wahi rows return hoti hain jaha condition True ho.

```
# Single condition
bios.loc[bios["height_cm"] > 215]

# Multiple conditions — & (AND), | (OR) use hota hai, 'and'/'or' NAHI
bios[(bios['height_cm'] > 215) & (bios['born_country'] == 'USA')]

# String contains check
bios[bios['name'].str.contains("keith", case=False)]

# Regex-based filter (advanced string matching)
bios[bios['name'].str.contains(r'^[AEIOUaeiou]', na=False)]

# isin() — multiple values check karne ke liye (SQL ke IN jaisa)
bios[bios['born_country'].isin(["USA", "FRA", "GBR"])]

# Combine karke bhi kar sakte ho
bios[bios['born_country'].isin(["USA", "FRA", "GBR"]) &
(bios['name'].str.startswith("Keith"))]
```

> **Analogy:** Har condition `(bios['height_cm'] > 215)` internally ek True/False ki Series banati hai jiski length same hai jitni rows DataFrame me hain. Jab tu `df[condition]` likhta hai, Pandas sirf woh rows deta hai jaha value True hai — bilkul jaise ek chalni (sieve) se sirf matching cheez chan ke nikaal lena.

`query()` method se bhi likh sakte ho (zyada readable lagta hai kabhi kabhi):

```
bios.query('height_cm > 215 and born_country == "USA"')
```

7. Columns Add Karna Aur Remove Karna

```
# Simple constant column add
coffee['price'] = 4.99

# Conditional column — np.where() Excel ke IF() formula jaisa
coffee['new_price'] = np.where(coffee['Coffee Type'] == 'Espresso', 3.99, 5.99)

# Column drop karna
coffee.drop(columns=['price'], inplace=True)

# Rename karna
coffee.rename(columns={'new_price': 'price'}, inplace=True)

# Existing columns se naya column derive karna
coffee['revenue'] = coffee['Units Sold'] * coffee['price']
```

`np.where(condition, value_if_true, value_if_false)` — yeh ek vectorized IF-ELSE hai. Ek-ek row pe Python loop chalane se yeh bahut fast hai kyunki Pandas/NumPy pura column ek saath process karte hain (C-level optimized).

`apply()` — jab logic complex ho aur np.where kaafi na ho:

```
bios['height_category'] = bios['height_cm'].apply(
    lambda x: 'Short' if x < 165 else ('Average' if x < 185 else 'Tall')
)
```

`apply()` ek function ko har row/value pe chalata hai (jaise Python's map() but Pandas-flavoured). Slower hai np.where se, but zyada flexible — complex custom logic likh sakte ho.

`inplace=True` ka matlab: original DataFrame ko directly modify karo, naya object return mat karo. Agar inplace nahi doge, toh original DataFrame same rahega aur tumhe naya assign karna padega (`coffee = coffee.drop(...)`).

8. Merging Aur Concatenating (Multiple Tables Combine Karna)

Yeh SQL ke JOIN jaisa concept hai.

```
# Merge — do DataFrames ko common column ke basis pe join karna
nocs = pd.read_csv('./data/noc_regions.csv')
bios_new = pd.merge(bios, nocs, left_on='born_country', right_on='NOC', how='left')
```

`how='left'` ka matlab: left DataFrame (bios) ki saari rows rakho, right (nocs) se match milne pe uski info attach kar do — na milne pe NaN aa jayega. (Options: left, right, inner, outer — bilkul SQL JOIN types jaisa.)

```
# Concat — DataFrames ko upar-neeche (ya side-by-side) jodna, join ke bina
usa = bios[bios['born_country'] == 'USA'].copy()
gbr = bios[bios['born_country'] == 'GBR'].copy()
new_df = pd.concat([usa, gbr])
```

> **Analogy:** `merge` do tables ko common key ke basis pe horizontally jodta hai (jaise VLOOKUP), jabki `concat` bas tables ko stack kar deta hai (ek ke neeche ek, same columns hone chahiye).

`.copy()` use karne ka reason: filter karke jab naya DataFrame banate ho, woh kabhi kabhi original ka "view" hota hai — `.copy()` explicitly bata deta hai ki yeh independent naya object hai, taaki baad me modify karte waqt "SettingWithCopyWarning" na aaye.

9. Missing/Null Values Handle Karna

```
# NaN ko specific value se fill karna
coffee['Units Sold'].fillna(0, inplace=True)

# Interpolate — pichli/agli value ke basis pe estimate nikalna (linear guess)
coffee['Units Sold'].interpolate(inplace=True)

# NaN wali rows drop karna
coffee.dropna(subset=['Units Sold'], inplace=True)

# Check karna kaha NaN hai
coffee.isna().sum()   # har column me kitne missing values hain
```

> **Analogy:** `fillna` ek fixed value se gap bharta hai (jaise Excel me "0" bhar dena), `interpolate` pattern dekh ke estimate lagata hai (jaise agar Monday=10 aur Wednesday=20 hai toh Tuesday ≈ 15 guess karega), aur `dropna` matlab jo row incomplete hai use hi hata do.

10. Aggregation — value_counts, groupby, pivot_table

Yeh Excel ke PivotTable jaisa concept hai — data ko summarize/group karke insights nikalna.

```
# value_counts — har unique value kitni baar aayi (frequency count)
bios['born_city'].value_counts()
# groupby — SQL "GROUP BY" jaisa: pehle group banao, phir aggregate karo
coffee.groupby(['Coffee Type'])['Units Sold'].sum()
coffee.groupby(['Coffee Type'])['Units Sold'].mean()

# Multiple grouping columns bhi ho sakte hain
bios.groupby([bios['year_born'], bios['month_born']])
['name'].count().reset_index().sort_values('name', ascending=False)
```

> **Analogy:** `groupby` teen step process hai — Split → Apply → Combine.
>
> **Split:** data ko category ke hisaab se buckets me todo (jaise Coffee Type = Espresso, Latte alag alag buckets).
> **Apply:** har bucket pe function chalao (sum, mean, count, etc.).
> **Combine:** results ko wapas ek table me jodo.

`.reset_index()` groupby ke baad kaam aata hai kyunki groupby result me grouping column index ban jata hai — reset_index() use wapas normal column bana deta hai taaki aage sorting/filtering aasan ho.

```
# Pivot table — rows/columns dono directions me summarize karna
pivot = coffee.pivot(columns='Coffee Type', index='Day', values='revenue')
```

> **Analogy:** Pivot table Excel ke PivotTable feature ki tarah hai — ek column ki values naye columns ban jati hain, aur cells me corresponding values fill ho jati hain. Jaise: rows = Days, columns = Coffee Type, cells = Revenue.

11. Advanced Functionality — Rolling, Cumsum, Rank, Shift

```
# Cumulative sum — running total banana
coffee['cumsum'] = coffee['Units Sold'].cumsum()

# Rolling window — moving average/sum jaisa concept (stock market charts me common)
latte = coffee[coffee['Coffee Type'] == "Latte"].copy()
latte['3day'] = latte['Units Sold'].rolling(3).sum()

# Rank — values ko rank dena (highest ya lowest se)
bios['height_rank'] = bios['height_cm'].rank(ascending=False)

# Shift — column ki values ko upar/neeche shift karna (previous/next period compare
karne ke liye)
coffee['yesterday_revenue'] = coffee['revenue'].shift(1)
```

> **Analogy:** `rolling(3)` ek "window" hai jo 3 consecutive rows ko dekh ke calculate karta hai, phir ek row aage khisak jata hai — jaise train ki 3 seats wali window jo dheere dheere aage badhti jaati hai. `shift(1)` matlab har row me "kal" (previous row) ki value laake current row ke saath rakh do — day-over-day comparison ke liye perfect.

12. Bade Datasets Ke Saath Kaam Karna (Chunks + PyArrow)

Jab dataset itna bada ho ki RAM me poora load nahi ho sakta, tab chunking use karte hain — file ko chhote-chhote parts me padhna, ek time pe ek chunk process karna.

```
for chunk in pd.read_csv('./data/results.csv', chunksize=1_000_000):
    # har chunk ek normal DataFrame hai, isi pe processing karo
    print(chunk.shape)
    # e.g. filter/aggregate karke result accumulate karo
```

> **Analogy:** Poori kitab ek saath yaad karne ki bajaye, chapter-by-chapter padhna — memory pe load kam padta hai.

PyArrow backend — Pandas ka naya, fast engine (especially strings ke liye):

```
results_arrow = pd.read_csv('./data/results.csv', engine='pyarrow',
dtype_backend='pyarrow')
results_arrow.info()
```

Yeh normal NumPy-backed Pandas se fast hota hai, especially bade text-heavy datasets pe, kyunki memory representation zyada efficient hai.

13. Data Ko Wapas File Me Save Karna

```
coffee.to_csv('./output/coffee_updated.csv', index=False)
coffee.to_excel('./output/coffee_updated.xlsx', sheet_name='Sheet1', index=False)
coffee.to_parquet('./output/coffee_updated.parquet')
```

`index=False` important hai — warna Pandas apna internal row-index bhi ek extra column ki tarah save kar dega, jo usually chahiye nahi hota.

Quick Cheat-Sheet Table

Category	Function/Method	Kya karta hai
Create	pd.DataFrame(data, columns=, index=)	Manually DataFrame banata hai
Create	pd.Series(data, name=)	Ek single column (Series) banata hai
Load	pd.read_csv(path)	CSV file load karta hai
Load	pd.read_excel(path, sheet_name=)	Excel file load karta hai
Load	pd.read_parquet(path)	Parquet (fast binary) file load karta hai
View	df.head(n) / df.tail(n)	Pehli/last n rows dikhata hai
View	df.sample(n)	Random n rows dikhata hai
Sort	df.sort_values(col, ascending=)	Column(s) ke hisaab se sort karta hai
Info	df.info()	dtype + non-null counts + memory
Info	df.describe()	Numeric summary stats
Info	df.shape / df.size	(rows, cols) / total elements
Info	df.nunique() / df['col'].unique()	Unique values count / list
Select	df.iloc[row_pos, col_pos]	Position-based selection (exclusive slicing)
Select	df.loc[row_label, col_label]	Label-based selection (inclusive slicing)
Filter	df[condition]	Boolean mask se rows filter karta hai
Filter	.str.contains()	String substring/regex match
Filter	.isin([...])	Multiple values match (SQL IN jaisa)
Filter	df.query('expr')	String expression se filter

Category	Function/Method	Kya karta hai
Column	df['new_col'] = ...	Naya column add karta hai
Column	np.where(cond, a, b)	Vectorized IF-ELSE column banata hai
Column	.apply(func)	Custom function har row/value pe chalata hai
Column	df.drop(columns=[...])	Column(s) hataata hai
Column	df.rename(columns={...})	Column naam badalta hai
Combine	pd.merge(df1, df2, on=, how=)	SQL-style JOIN
Combine	pd.concat([df1, df2])	DataFrames ko stack karta hai
Null	.fillna(value)	NaN ko value se bharta hai
Null	.interpolate()	Pattern se NaN estimate karta hai
Null	.dropna(subset=[...])	NaN wali rows hataata hai
Aggregate	.value_counts()	Har unique value ki frequency
Aggregate	.groupby(col)['x'].agg_func()	Group karke sum/mean/count nikalta hai
Aggregate	.pivot(index=, columns=, values=)	Excel PivotTable jaisa summary table
Advanced	.cumsum()	Running total
Advanced	.rolling(n).sum()/.mean()	Moving window aggregation
Advanced	.rank(ascending=)	Values ko rank deta hai
Advanced	.shift(n)	Column ko n positions shift karta hai
Large data	pd.read_csv(..., chunksize=n)	File ko chunks me padhta hai
Large data	engine='pyarrow', dtype_backend='pyarrow'	Fast PyArrow backend use karta hai
Save	.to_csv(path, index=False)	DataFrame ko CSV me save karta hai
Save	.to_excel(path, index=False)	DataFrame ko Excel me save karta hai
Save	.to_parquet(path)	DataFrame ko Parquet me save karta hai